**EXAMEN DE FIN D'ÉTUDES SECONDAIRES – Sessions 2024**

# QUESTIONNAIRE

| *Date :* | 23.05.24 | | *Horaire :* | 08:15 - 11:15 | *Durée :* | 180 minutes |
|---|---|---|---|---|---|---|
| *Discipline :* | SCIPR | *Type :* | écrit | *Section(s) :* | CI | |
| | | | | *Numéro du candidat :* | | |

# Modelling part (10 points)

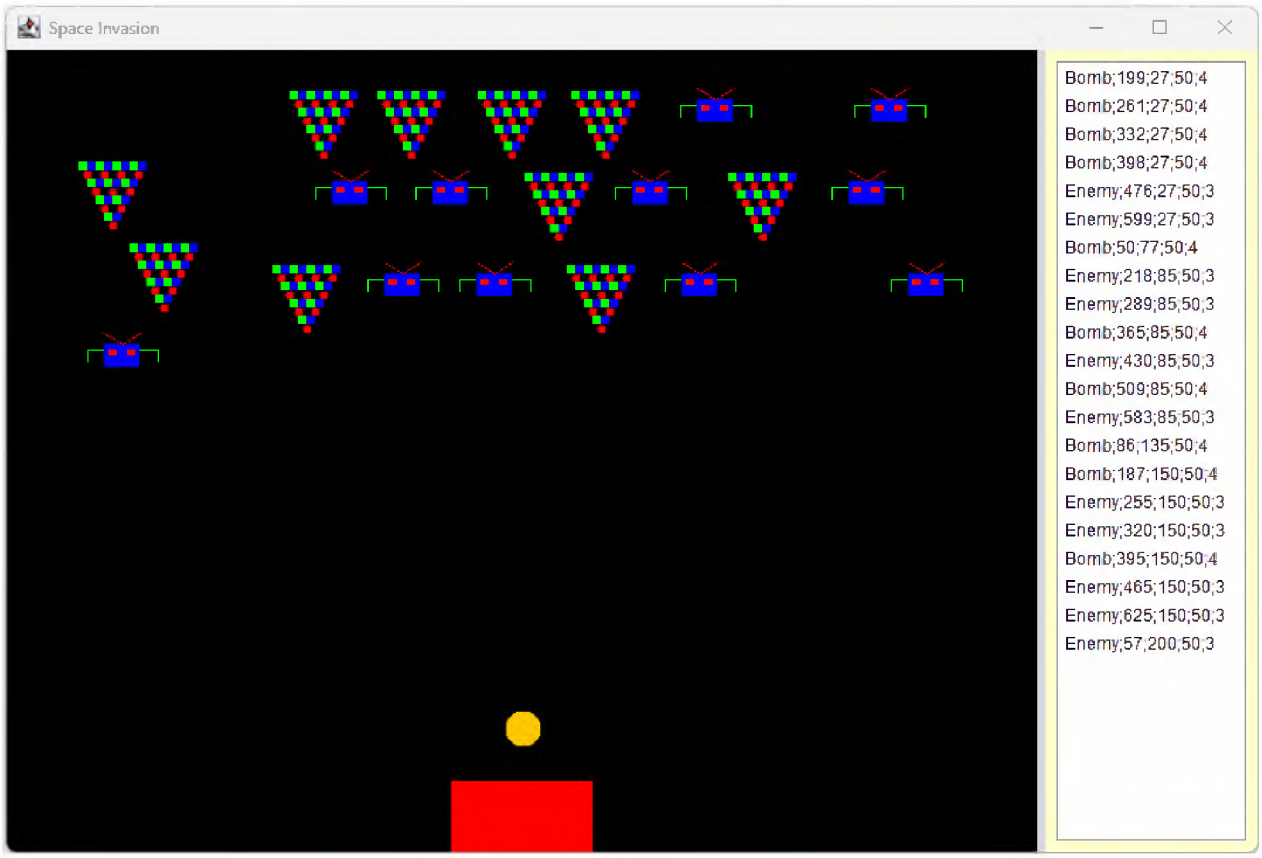
- The modelling part must be written on paper and must be handed in before the implementation part can start.
- The modelling part is due after 35 minutes at the latest, but you are free to hand it in earlier.

A game company has approached you to develop a new version of the classic game "Space Invasion". The game consists of various objects that move from left to right and up and down in space. These objects can be shot down by the player to prevent them from reaching and destroying the game. The player is represented by a red rectangle, which shoots bullets that move from bottom to top. The game ends when the player has no more lives or when all objects have been destroyed.

## Principle of the application

The game company provides you with a CSV file that they got from an original version of the game. In the first version there are four types of objects, called enemy, player, bomb, and bullet. Each object has a certain number of lives, an individual representation, a position, and a size that indicates the dimensions of the object. In addition to the size, the player and the enemy also have a height that determines its dimension. The company gives you the following additional information:

1. As soon as the game starts, all objects are loaded from the CSV file and positioned in the appropriate place.
2. The objects must be stored in a data structure that allows quick access to an element via an index.
3. If two objects collide with each other, both should lose a life.
4. If an object has no more lives or is off the screen, it should be deleted.
5. The player moves from left to right, the bullet only moves upwards, and the other objects move from left to right.
6. All objects should be written to a CSV file when the game is closed.




## Work to be done

Adding comments onto your diagram is recommended in order to explain your choice!

Draw a class model of the application model on paper using the usual UML conventions.

1. First add the required classes for the various space objects and their attributes.
2. Add a class that represents the space and choose a suitable data structure for storing the objects.
3. Add the methods required to save and read all objects in a CSV file.
4. Add the methods for drawing all objects in the space.
5. Add the methods for moving all objects in the space.
6. Add all methods that make it possible to recognise a collision of two objects.
7. Add all methods to delete an object that no longer has any life or is no longer on the screen.
8. You can add further explanations to your class model.

# Implementation part (50 points)

In your working directory (to be defined by each school), you will find a folder called EXAMEN_CI. Rename this folder by replacing the current name with your exam code (example of notation: LXY_CI2_05). All your files should be saved in this folder, which will be called "your folder" in the following.

You will programme an application that is a simplified version of the Space Invasion game. In the game there are various objects that move around in space. These can be shot down by a player so that they cannot attack the player. If the player has no more lives or all objects have been shot down, the game is over. Open the "Space" project in your folder. In the project folder you will also find a CSV file called "space.csv", which you can use as an example to test your implementation. The player can move left or right by pressing the arrow keys and fire a bullet using the space bar. The right-hand panel can be displayed or hidden using the "d" button. Complete the application based on the executable version, the UML diagram and the instructions given later.

## The class "SpaceObject" 6 points

Complete the abstract class **SpaceObject**, which represents an object in the game that is a square by default. The class has the following attributes:

- **x, y** represents the top left corner of the object
- **lives** correspond to the number of lives the object still has
- **size** corresponds to the size of each side of the object

The **constructor** initialises the value of **x**, **y** and **size** using the corresponding parameters. The number of lives is **1** by default. Add the methods as described in the UML schema. 0.25p

The class has the following methods:

- The **move** method moves the object from left to right by one pixel. If the object has completely exceeded the right edge of the area, the object should reappear at the left edge, but moved down by **size** pixels. The object should not be visible immediately but should appear step by step at the left edge. (See the demonstration programme in your folder). 2p
- The method **getRectangle** returns the current object as a **Rectangle**[1], with the position **x,y** and the **size** as width and height. 0.5p
- The **checkAndHandleCollision** method checks whether the current space object collides with another space object. For the simplicity of the method, only rectangles of the individual objects are compared with each other and not the exact outlines of the figure. In the case of a collision, both objects lose a life. You may use the **intersects** method from the **Rectangles** class. 1.5p
- The **isObsolete** method specifies whether the space object is obsolete or not. The object is obsolete when it has no more lives or has left the upper or lower area. 0.5p
- The **draw** method is abstract. 0.25p

[1] See the attached javadoc pdf for more details (**java.awt.Rectangle**)

- The **toString** method returns the textual representation of the space object in CSV format, which corresponds to the following format:
  **class name;x-coordinate; y-coordinate;size;lives.** 1p

> **Note:** Use the code **getClass().getSimpleName()** to retrieve the name of the class whose object has been instantiated.

## The class "Bullet" 2 points

This class is a subclass of the **SpaceObject** class and draws a bullet. A bullet has only one life and is drawn as an orange-coloured filled circle that with a diameter given by the attribute **size**. Add the constructor and the methods as described in the UML diagram.

- The **draw** method draws the bullet. 0.25p
- The **move** method moves the bullet up by one pixel. 0.25p
- The **checkAndHandleCollision** only checks whether two object collide if the objects specified as parameter is not of the type **Bullet**. 1.5p

## The class "Enemy" 8.5 points

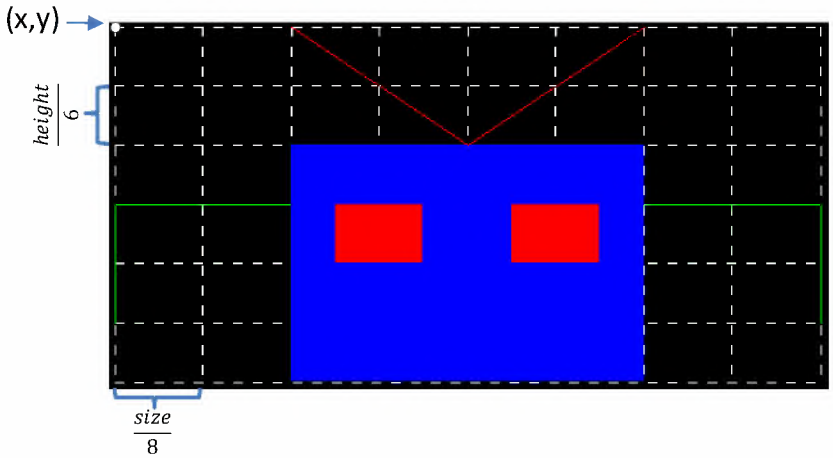
The Enemy class stands for an enemy in the game and is a subclass of the **SpaceObject**.

The class has the following attribute:

- **height** corresponds to the height of the enemy object

An enemy always has three lives, the **height** is half the **size**, and the **size** stands for the width of the enemy. 0.25p

- The method **getRectangle** returns the current object as a **Rectangle**, with the position **x, y** and the **size** as width and **height** as height. 0.25P
- The **draw** method draws the enemy as shown in the figure below. The drawing must adapt dynamically to the **size** and **height**. The enemy's arms are green, the eyes and antennas are red, and the body (rectangle) is blue. The width of an eye is one eight of the total width and the height of an eye is one sixth of the total height. Use the private methods **drawAntennas** (color = red), **drawEyes** (color = red), **drawBody** (color = blue), and **drawArms** (color = green), to draw the respective objects of the enemy. Afterwards, call these in the **draw** method. (The dashed white lines are only a help and do not have to be drawn). 8p



## The class "Player" 2 points

The Player class represents one player in the game and is a subclass of the **SpaceObject**. A player is represented by a red square and has 4 lives. 0.25p

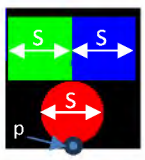
The class has the following methods:

- The **draw** method draws the player with a red filled rectangle. 0.25p
- The **move** method moves the object horizontally by the value specified by the parameter **steps.** 0.5p
- With the help of the **shoot** method, a new **bullet** is created, which has the centre of the player as its centre and is located directly above the player. The size of the bullet is one quarter of the **size** of the player. 1p

## The class "Bomb" 6.5 points

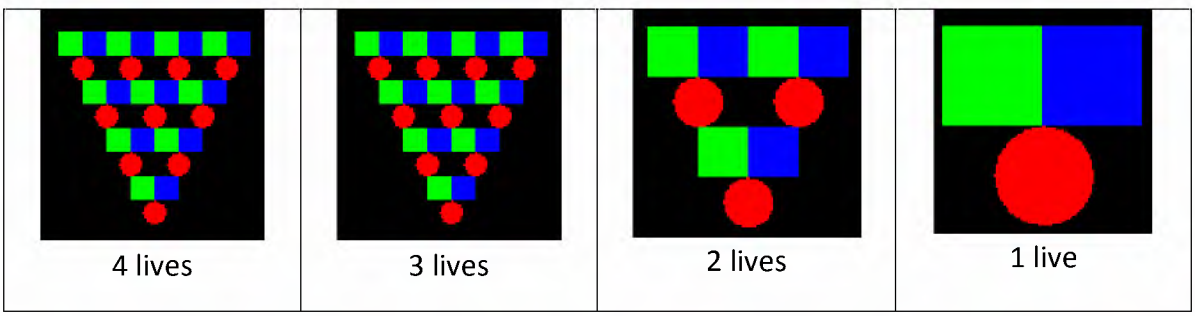
The **Bomb** class represents a bomb and is a subclass of **SpaceObject**. A bomb is represented by a fractal and has 4 lives. 0.5p

- The **drawShapes** method draws a filled green square (top left) and a filled blue square (top right), both directly above a centered filled red circle. The parameter **p** specifies the lowest point of the red cercle, and the parameter **s** specifies the size of the shapes.




2p

- The <u>private</u> **draw** method is used to draw recursive the bomb.The parameter **p** specifies the position of the three shapes, the parameter **s** specifies the size of the shapes and **n** specifies the remaining recursive calls. If **n** is less than 1, the recursive calls end. Look at the examples an code it accordingly. 3.5p




- The <u>public</u> **draw** method specifies the start values for the recursive calls. The size of all shapes is $\frac{size}{2*lives}$ and the complexity of the bomb depends on the number of lives it has. 0.5p

## The class "Node"

The **Node** class is provided. It represents a node in a linked list in which a **SpaceObject** can be stored.

## The class "SpaceInvasion" 25 points

The **SpaceInvasion** class is partially provided and must be completed. It contains all space objects in the game, stores them in a linked list and allows them to be read from a CSV file. The class has the following attributes:

- **root** represents the first node in the list
- **size** is the size of the list
- **player** represents the player in the game

The constructor places the player so that the centre of it is horizontally in the middle of the screen. The y-coordinate of the player corresponds to half the size of the player away from the bottom. The size of the player is 100 pixels. 0.5p

The class has the following methods:

- The **add** method adds a new space object at the end of the list. The **size** is increased by one. 1.5p
- The **toArray** method converts the linked list into an array. All objects of the linked list are placed in an array in the same order. 3p
- The **gameOver** method indicates whether the game is over. This is the case if the player has no more lives or if the list is empty. 0.5p
- The **drawAll** method draws the entire game. If the game is over, "Game Over" is written in white at the position (100,100). In the other case, the game and all objects are drawn. 3p
- The **moveAll** method moves all the objects in the game. 1p
- The **checkHit** method checks and handles whether objects collide with each other or with the player. Make sure that the objects are not compared with themselves and that they are not checked twice! 5p
- The **movePlayer** method moves the player by the specified parameter **steps**. 0.25p
- The **shoot** method is used to shoot a bullet from the player and add it to the list. 0.25p
- In the **removeAllObsolete** method all objects that are obsolete are deleted. 6p
- The **loadFromFile** method reads all space objects from a CSV file and adds them to the list. The name of the file is specified by the **fileName** parameter. All possible exceptions are passed on to the calling class.
  The format of one line corresponds to the following format:
  **class name;x-coordinate;y-coordinate;size**
  In the case that a space object is read whose type does not exist, an exception of type **IllegalArgumentException** is to be thrown with a meaningful text.
  **Note:** the player is never part of the CSV file. 4p

## The class "DrawPanel"

The **DrawPanel** class is provided. The class draws the game and has a **spaceInvasion** attribute with the corresponding manipulator.

## The class "MainFrame"

The **MainFrame** class is provided. The class has the **spaceInvasion** attribute, which represents the game. When the programme starts, all objects are loaded from the "space.csv" file in your folder.

## UML of the model